Validación de NIF/CIF/VAT de HubSpot contra VIES

Documentación del proyecto · Actividad Vexco · Joan · 29 de octubre de 2026

1. Qué hace el script y cómo lo he planteado

El proyecto es un script en PHP que recorre las empresas de la cuenta Demo de HubSpot que tienen informada la propiedad `nif_cif_vat`, valida cada número contra la API pública VIES de la Comisión Europea y guarda el resultado en la propia empresa, en tres propiedades nuevas creadas por API. Al final imprime un resumen por consola (total, válidas, inválidas, con error y omitidas).

Lo he dividido en dos comandos independientes y en clases con una única responsabilidad:

1. `php setup.php` crea por API el grupo de propiedades **Ejercicio Vexco – Joan** y las propiedades `vies_joan_estado`, `vies_joan_fecha` y `vies_joan_motivo` (texto de una línea). Si ya existen, lo omite con HTTP 409, así que puede ejecutarse varias veces.
2. `php run.php` lee las empresas con búsqueda filtrada `HAS_PROPERTY` sobre `nif_cif_vat`, pagina con `after` de 100 en 100 y las ordena por `hs_object_id` para que el recorrido sea estable. Para cada una normaliza el dato, decide si se valida u omite, consulta VIES y acumula el resultado.
3. Los resultados se envían a HubSpot con `batch/update` en lotes de 100 y, al terminar, se imprime el resumen.

Archivo	Funciones
`setup.php` · `SchemaSetup.php`	Crea el grupo y las tres propiedades por API.
`run.php` · `Processor.php`	lee, decide, valida, escribe por lotes y muestra el resumen.
`HubSpotClient.php`	Cliente HTTP (Guzzle): búsqueda paginada, batch update y reintentos ante 429, 423 y 5xx.
`VatParser.php`	Normaliza el dato y decide si se valida con VIES o se omite.
`ViesClient.php`	Llama a VIES y traduce la respuesta a VALIDO, INVALIDO o ERROR_TEMPORAL.
`Config.php`	Carga el token desde `.env` (nunca se sube al repositorio).
`test_vies.php`	Prueba manual del parser y de VIES con casos reales y de borde.

Idempotencia. Antes de procesar una empresa se lee su `vies_joan_estado`. Si ya es `VALIDO`, `INVALIDO` u `OMITIDO`, se cuenta en el resumen pero no se vuelve a consultar ni a escribir. Solo se reprocesan las empresas sin estado y las que están en `ERROR_TEMPORAL`, de modo que una segunda ejecución no repite trabajo ni pisa resultados.

Datos de partida intactos. El script solo escribe en las tres propiedades `vies_joan_*`. Nunca modifica `nif_cif_vat` ni ninguna otra propiedad, y no usa ninguna otra propiedad de nombre parecido.

Resultado de mi ejecución en la cuenta Demo:

Concepto	Nº de empresas
Total procesadas (con `nif_cif_vat`)	83
Válidas (VALIDO)	68
Inválidas (INVALIDO)	4
Con error (ERROR_TEMPORAL)	0
Omitidas (OMITIDO)	11

2. Scopes de la service key y por qué

He usado una **service key.** Solo he añadido cuatro scopes:

Scope	Para qué lo necesito
`crm.objects.companies.read`	Leer las empresas y su `nif_cif_vat`, además de los estados ya guardados (idempotencia).
`crm.objects.companies.write`	Guardar el resultado en las propiedades `vies_joan_*` mediante `batch/update`.
`crm.schemas.companies.read`	Consultar el esquema de propiedades de empresas.
`crm.schemas.companies.write`	Crear por API el grupo y las tres propiedades, como pide el enunciado.

Solo he añadido estas, ya que en el proceso de plantearme el proyecto, solo vi necesario estos 4.

3. Tratamiento de datos mal formados y errores de VIES

3.1 Datos mal formados

`VatParser` pasa el valor a mayúsculas y elimina todo lo que no sea letra o número (espacios, puntos, guiones, corchetes). Después decide:

Caso	Ejemplos	Resultado
Vacío o solo espacios	`""`, `"  "`	OMITIDO (Dato vacío)
Valor basura	`N/A`, `PENDIENTE`, `12345`	OMITIDO (No hay nada que validar)
Sin prefijo de país	`A82018474`, `500697256`	OMITIDO (No hay codigo de pais)
País fuera de VIES	`GB123456789`, `CHE-123.456.789 MWST`	OMITIDO · (Países fuera de la UE, por tanto no admitidos por VIES)
Prefijo UE, con ruido de formato	`[ ES A39000013 ]`, `DE 129 274 202`, `ES.A48265169`	Se normaliza y se envía a VIES

Los datos mal formados nunca se marcan como inválidos: se omiten con su motivo, porque no se ha podido comprobar nada contra VIES. `INVALIDO` queda reservado para los números que VIES rechaza expresamente.

3.2 Respuestas y errores de VIES

Respuesta de VIES	Estado	Qué se guarda
`valid: true`	VALIDO	Fecha en ISO 8601 (`date('c')`); motivo vacío.
`valid: false`	INVALIDO	Motivo devuelto por VIES (`userError` o `error`) o un texto genérico; fecha vacía.
Timeout, sin conexión, HTTP 5xx o 429	ERROR_TEMPORAL	Motivo descriptivo. No se marca como inválida.
Cuerpo vacío, JSON ilegible, `actionSucceed: false` o respuesta inesperada	ERROR_TEMPORAL	Motivo descriptivo. No se marca como inválida.

Una empresa en `ERROR_TEMPORAL` no se considera resultado definitivo: en la siguiente ejecución se vuelve a consultar, y es lo único (junto con las empresas sin estado) que se reprocesa.

3.3 Errores y rate limit de HubSpot

`HubSpotClient` reintenta hasta 5 veces ante HTTP 429, 423 (bloqueo por muchas escrituras seguidas) y 5xx, y

también ante errores de conexión. Si HubSpot envía `Retry-After` lo respeta; si no, usa espera exponencial con un mínimo de 2 s y un máximo de 30 s. La paginación sigue `paging.next.after` hasta que no hay más páginas, y se usa un generador para no cargar todas las empresas en memoria.

4. Dudas y decisiones

Dudas y decisiones

- **¿Hace falta schemas.write?** Sí, mientras el propio script cree grupo y propiedades por API. Lo dejé así para poder crearlos desde API como pide el enunciado.
- **Versión de la API.** La documentación actual usa rutas con fecha (`2026-09`), pero `v3` funciona en la cuenta y es la que he usado. La ruta base está en una constante de `HubSpotClient` para poder cambiarla si es necesario desde un unico punto.
- **Error 423 de HubSpot.** Al leer la documentacion de la API, vi que HubSpot puede responder este codigo de error, que , según lo visto, es un bloqueo temporal por parte de hubspot, asi que el script esperarà para poder reintentarlo
- **Escritura por lotes.** Uso `batch/update` de 100 en 100 para reducir llamadas y presión no arriesgar a sobrepasar el rate limit
- **Qué se guarda en cada estado.** Solo `VALIDO` lleva fecha; en `VALIDO` limpio el motivo y en `INVALIDO` limpio la fecha, para no dejar valores de ejecuciones anteriores.